

DOKUMENTASI TEKNIS SISTEM

Sistem Pendukung Keputusan & Kecerdasan Buatan (AI) BathariSri

1. Ikhtisar Sistem (System Overview)

BathariSri adalah platform aplikasi web berarsitektur *Software Engineering* modern yang menggabungkan Sistem Pendukung Keputusan (*Decision Support System / DSS*) dengan Kecerdasan Buatan (*Artificial Intelligence / AI*). Aplikasi ini dirancang untuk memandu siklus hidup pertanian padi secara *end-to-end*; mencakup pendataan lahan, pemindaian penyakit via citra daun (*Computer Vision*), perhitungan dosis pupuk logis (*Fuzzy Mamdani*), estimasi panen (*Multiplicative Regression*), hingga alokasi limbah organik.

Dokumen ini memuat spesifikasi teknis arsitektur, basis data, alur algoritma, dan dokumentasi instruksi sistem.

2. Arsitektur & Teknologi

Sistem ini menggunakan arsitektur API-Driven Microservices untuk memisahkan beban lalu lintas (*traffic*) pengguna web dengan beban komputasi *Deep Learning*.

- Antarmuka (Frontend): React.js & Tailwind CSS. Dihubungkan menggunakan Inertia.js untuk menciptakan pengalaman *Single Page Application* (SPA) tanpa *loading* ulang halaman.
- Orkestrator Utama (Main Backend): Laravel 13 (PHP). Mengelola manajemen *state*, autentikasi, relasi basis data (MySQL/MariaDB), dan eksekusi algoritma *Fuzzy* serta Regresi.

- Layanan Cerdas (AI Microservice): FastAPI (Python). Menangani pemrosesan citra (ResNet-50) secara independen pada *port* yang terpisah dari *server* utama.

3. Skema Basis Data

Skema *database* BathariSri mengimplementasikan relasi terstruktur (*One-to-Many*) untuk menjamin tidak terjadinya redudansi dan tercampurnya privasi data antar-petani.

3.1 Entity-Relationship Diagram (ERD)



3.2 Spesifikasi Tabel Basis Data

Terdapat 14 Entitas Tabel yang terbagi ke dalam dua klaster fungsi utama:

A. Klaster Transaksional & Riwayat Pengguna

1. `users` (PK: `id`): Autentikasi dan identitas aktor.
2. `lahans` (FK: `user_id`): Profil aset tanah petani.
3. `planting_schedules` (FK: `lahan_id`): Riwayat kalender tanam.
4. `disease_scans` (FK: `lahan_id`): Riwayat pemindaian penyakit oleh AI.
5. `spk_fertilizer_recs` (FK: `lahan_id`): Riwayat keluaran komputasi resep pupuk.
6. `harvest_predictions` (FK: `lahan_id`): Riwayat tebakan kalkulasi tonase panen.
7. `waste_recommendations` (FK: `harvest_id`): Hasil instruksi tata kelola limbah.

B. Klaster Master Referensi (Dikelola Admin)

1. `disease_refs`: Kamus penyakit dan instruksi PPL.
2. `variety_refs`: Kamus potensi dasar varietas padi.
3. `fertilizer_refs`: Indeks harga pupuk terkini.
4. `commodity_prices`: Indeks harga jual beras gabah.
5. `waste_price_refs`: Valuasi tukar limbah.
6. `spk_weight_configs`: Ambang batas (*threshold*) & persentase denda algoritma.
7. `articles`: Pangkalan data berita edukasi.

4. Alur Kerja Algoritma (Workflow)

4.1 Eksekusi Sistem (Sisi Petani)



Logika pemrograman aplikasi bekerja melalui 5 siklus berurutan:

1. Inisiasi Lahan: Pengikatan data luas dan umur padi (HST).
2. AI Computer Vision: Klasifikasi ResNet-50. Tingkat keparahan (*Severity*) diputuskan melalui validasi bertingkat: *Severe* (Confidence $\geq 85\%$), *Moderate* ($\geq 65\%$), dan sisanya *Mild*.
3. SPK Fuzzy Mamdani: Menghasilkan nilai absolut (gramase pupuk) dari variabel linguistik luas lahan dan tingkat penyakit.
4. Regresi Panen: Menghitung sisa tonase akibat variabel penalti/denda dari penyakit.
5. Rule-Based Limbah: Mengevaluasi ketersediaan kandang/lahan kosong untuk mencabangkan *output* akhir menjadi Pakan Ternak, Kompos, atau Briket pembakaran.

4.2 Pembaruan Referensi (Sisi Administrator)



Admin berinteraksi secara dinamis dengan sistem melalui antarmuka dasbor untuk memperbarui bobot konfigurasi. Pembaruan tabel referensi akan langsung berimplikasi pada *output* komputasi di *frontend* petani secara *real-time*.

5. Interaksi Sistem

5.1 Use Case System (Wewenang Aktor)



Aktor sistem dipisahkan secara ketat. Petani bertindak sebagai eksekutor fasilitas lahan dan pemantik modul prediksi (AI & SPK). Sebaliknya, Administrator tidak memiliki akses ke fitur prediksi, namun memiliki kendali *Read/Write* mutlak ke dalam Tabel Master dan Sistem *Early Warning* Wabah Nasional.

5.2 Activity Diagram (Logika Lintas Layanan)



Alur komunikasi jaringan digambarkan melalui sistem lempar-tangkap (*Ping-pong*) lintas partisi (*Swimlane*). Transmisi *file* gambar dilempar menuju partisi Python (kanan), namun hasil *Confidence* dan kelas penyakitnya akan dikembalikan ke partisi PHP (tengah) untuk dilanjutkan menuju komputasi SPK Pupuk dan Panen yang memerlukan koneksi basis data.

6. Panduan Penggunaan Web

6.1 Instruksi Eksekusi Bagi Petani

1. Petani membuat akun pada halaman pendaftaran.
2. Navigasi ke opsi "Lahan Anda" $\rightarrow$ "Tambah Lahan". Input luas area dan pilih varietas.
3. Untuk *Scan* Penyakit, arahkan ke menu "Diagnosis AI", unggah citra daun, lalu amati tingkat keyakinan model.
4. Lanjutkan alur dengan menekan "Hitung Rekomendasi Pupuk". Sistem akan mentransmisikan data penyakit sebelumnya tanpa perlu *input* manual.
5. Akhiri pada menu "Prediksi Panen". Centang fasilitas limbah desa untuk mengetahui kalkulasi ekonomi hasil buangan. (*Catatan: [Sisipkan Screenshot Halaman Frontend Petani Di Sini]*)

6.2 Instruksi Monitoring Bagi Administrator

1. Masuk (*Login*) menggunakan *email* khusus admin yang telah disemai (*seeded*).
2. Tampilan awal akan menyajikan Dasbor Analitik Nasional (grafik persebaran penyakit terbanyak).

3. Jika *Alert* menyala, navigasikan ke menu pemantauan untuk menginstruksikan PPL turun ke lapangan yang berstatus *Severe*.
4. Gunakan menu Manajemen Harga & Konfigurasi untuk memanipulasi algoritma. (Catatan: [Sisipkan Screenshot Dasbor Admin Di Sini])

7. Deployment & Konfigurasi Lokal

Agar aplikasi dapat dijalankan dari *source code repository*, administrator lokal harus menyalakan dua buah *server* terpisah:

7.1 Spesifikasi Server AI (Microservice)

Server perantara (*API Bridge*) yang dikomunikasikan secara internal (tidak terekspos ke *public client*):

- Request URL: `http://localhost:5000/api/predict`
- Method: `POST multipart/form-data`
- Command Jalan (Python): `uvicorn main:app --port 5000`

7.2 Spesifikasi Server Utama (Laravel/React)

Menjalankan mesin utama dan antarmuka web:

1. Eksekusi `composer install` dan `npm install`.
2. Sesuaikan berkas `.env` ke *database* (`DB_DATABASE=bathari_sri`).
3. Jalankan rilis tabel dasar: `php artisan migrate --seed`.
4. Nyalakan antarmuka klien via *Vite*: `npm run dev`.
5. Nyalakan *server* inti via *PHP*: `php artisan serve`.
6. Buka aplikasi di `http://localhost:8000`.